Sprint 05 - useState, Rendering, Immutable State Updates, and Beginner useReducer.

0. Where You Are Before Sprint 05 (Sprint 05 වලට කලින් ඔයා දැනගත යුතු දේවල්)

- **Concept Explanation (Sinhala):** Sprint 05 පටන් ගන්න කලින් ඔයා පහත තියෙන React මූලික සංකල්ප ටික හොඳට තේරුම් අරන් ඉන්න ඕනේ:
  - React වලින් user interfaces (UI) හදන්නේ **components** පාවිච්චි කරලා.
  - JavaScript ඇතුළේ UI එක ලියන්න අපිට උදව් වෙන්නේ **JSX** කියන syntax එක.
  - Parent component එකක ඉඳන් child component එකකට data පාස් කරන්නේ **Props** හරහායි.
  - Component එකක් ඇතුළට තවත් JSX කොටසක් wrap කරලා යවන්න **children** prop එක පාවිච්චි කරන්න පුළුවන්.
  - Array එකක තියෙන elements screen එකේ list එකක් විදිහට පෙන්වන්න (render කරන්න) **map()** function එක පාවිච්චි කරනවා.
  - React එකට list එකක items නිවැරදිව track කරගන්න උදව් වෙන්නේ හැම තිස්සෙම දෙන unique **stable keys** මඟිනුයි.
  - Button clicks, typing, සහ form submissions වගේ දේවල් වලට React ප්‍රතිචාර දක්වන්නේ **Events** හරහායි.
  - HTML forms වල data React state එකේ store කරලා තියාගන්න **Controlled inputs** පාවිච්චි කරනවා.
  - Form එකක් submit වෙද්දී මුළු page එකම reload වෙන එක නතර කරන්නේ **event.preventDefault()** එකෙන්.

  මේ Sprint 05 කියන්නේ ඔයාගේ React App එක ඇත්තටම **state-driven** (state එක මත ධාවනය වන) එකක් බවට පත්වෙන වැදගත්ම සන්ධිස්ථානයයි. මීට කලින් ඔයාට පුළුවන් වුණේ data පෙන්වන්න විතරයි, නමුත් මේ පාඩමෙන් පස්සේ ඔයාගේ app එකට data මතක තියාගන්නත්, screen එක නිවැරදිව update කරන්නත්, mutation bugs මඟහැර ගන්නත් පුළුවන් වෙනවා.

1. Sprint 05 Goal (Sprint 05 හි ඉලක්කය)

- **Concept Explanation (Sinhala):** මේ පාඩමේ ප්‍රධානම අරමුණ වෙන්නේ React කොහොමද data මතක තියාගන්නේ (remember) සහ UI එක update කරන්නේ කියලා තේරුම් ගන්න එකයි.

  පාඩම අවසානයේ ඔයාට:
  - Dynamic UI එකක් හදන්න **useState** පාවිච්චි කරන්න,
  - **State** කියන්නේ මොකක්ද කියලා විස්තර කරන්න,
  - React වල **render cycle** එක පැහැදිලි කරන්න,
  - **Trigger, render, සහ commit** කියන පියවර තුන පැහැදිලි කරන්න,
  - හැම component instance එකකටම වෙන වෙනම **private state** එකක් තියෙන බව තේරුම් ගන්න,
  - React **Hooks** හැමවෙලේම component එකක top level එකේ විතරක් call කරන්න පුරුදු වෙන්න,
  - පරණ state එක (previous state) පදනම් කරගෙන අලුත් state එක update කරන්න,
  - Object state එකක් update කරද්දී එහි තියෙන වෙනත් fields නැති කරගන්නේ නැතුව update කරන්න (using spread),
  - Array state එකක් update කරද්දී mutation කරන්නේ නැතුව (without mutation) කරන්න,
  - Spread operator පාවිච්චි කරලා array එකකට අලුත් items එකතු කරන්න,
  - **filter()** පාවිච්චි කරලා items ඉවත් කරන්න,
  - **map()** පාවිච්චි කරලා array එකක items update කරන්න,
  - Direct mutation (කෙලින්ම state වෙනස් කිරීම) නිසා bugs ඇති වෙන්නේ ඇයි කියලා පැහැදිලි කරන්න,
  - Sibling components දෙකක් අතර data share කරන්න ඕන වුණාම **lifting state up** කරන්න,
  - Beginner මට්ටමේ **useReducer** භාවිතය තේරුම් ගන්න,
  - `useState` වෙනුවට `useReducer` පාවිච්චි කිරීම වඩාත් පැහැදිලි වෙන්නේ කොයි වගේ වෙලාවටද කියා ඉගෙන ගන්න,
  - සම්පූර්ණ **Packing List App** එකක් නිර්මාණය කරන්න පුළුවන් වෙනවා.

2. Big Mental Model (ප්‍රධාන මානසික ආකෘතිය)

- **Concept Explanation (Sinhala):** React app එකක් වැඩ කරන්නේ මෙන්න මේ සරල cycle එකටයි:

  $$\text{User Action (Button Click)} \rightarrow \text{State Changes} \rightarrow \text{React Re-renders Compon}$$

  උදාහරණයක් විදිහට, user කෙනෙක් button එකක් click කරාම `setCount` run වෙලා `count` state එක වෙනස් වෙනවා. එතකොට React මුළු component එකම ආයෙත් render කරලා අලුත් count එක screen එකේ පෙන්වනවා.

3. What Is State? (State කියන්නේ මොකක්ද?)

- **Concept Explanation (Sinhala): State** කියන්නේ React component එකක් ඇතුළේ මතක තියාගන්නා (remember), කාලයත් සමඟ වෙනස් විය හැකි data වලටයි. සරලවම කිව්වොත්, **State යනු වෙනස් විය හැකි data සඳහා React සතු මතකයයි (memory).**

  State වලට උදාහරණ:
  - Counter number එකක්
  - Form input එකක type කරන දේවල්
  - Cart එකේ තියෙන බඩු (cart items)
  - Log වෙලා ඉන්න user ගේ තොරතුරු
  - Dark/Light theme mode එක
  - Select වෙලා තියෙන tab එක
  - Todo list එකක්

  ### Props vs State (වෙනස හඳුනාගනිමු)

Props	State
Component එකකට පිටතින් ලැබෙන data (Read-only)	Component එක සතු සහ එය විසින්ම වෙනස් කරගන්නා data (Mutable inside)

- **Code Block:**

```
// Example of Props:
function UserCard({ name }) {
  return <h2>{name}</h2>;
}

// Example of State:
const [count, setCount] = useState(0);
```

4. Basic useState (මූලික useState භාවිතය)

- **Concept Explanation (Sinhala):** `useState` කියන්නේ React Hook එකක්. මෙයින් component එකකට වෙනස් වෙන data මතක තබා ගැනීමේ හැකියාව ලැබෙනවා.

  ලියන ආකාරය: `const [value, setValue] = useState(initialValue);`
  - `value` : දැනට තියෙන state අගය.
  - `setValue` : state එක update කරන්න පාවිච්චි කරන function එක (setter function).
  - `initialValue` : state එක පටන් ගන්නකොට තියෙන පළමු අගය.

5. Basic Counter Example (සරල කවුන්ටරයක උදාහරණයක්)

- **Concept Explanation (Sinhala):** පහත කේතයෙන් සරල Counter component එකක් නිර්මාණය කරලා තියෙනවා. මෙහි `count` එක 0න් ආරම්භ වී, button එක click කරන හැම වෙලාවකම `setCount(count + 1)` මඟින් එය 1කින් වැඩි වෙනවා.
- **Code Block:**

```
import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  function handleIncrease() {
    setCount(count + 1);
  }

  return (
    <main>
      <h1>Counter</h1>
      <p>Count: {count}</p>
      <button type="button" onClick={handleIncrease}>
        Increase
      </button>
    </main>
  );
}
```

6. Do Not Update State Like a Normal Variable (State එක සාමාන්‍ය variable එකක් වගේ update කරන්න එපා!)

- **Concept Explanation (Sinhala):** React වලදී කිසිම වෙලාවක `count = count + 1` වගේ කෙලින්ම state update කරන්න බැහැ.

  ඇයි ඒ? මොකද සාමාන්‍ය variable එකක් වෙනස් කරාම React දැනගන්නේ නැහැ screen එක update (re-render) කරන්න ඕනේ කියලා. Screen එක update කරන්න නම් අනිවාර්යයෙන්ම setter function එක (`setCount`) හරහාම වෙනස් කිරීම කරන්න ඕනේ.

7. React's Render Cycle (React හි Rendering ක්‍රියාවලිය)

- **Concept Explanation (Sinhala):** State එකක් හෝ Prop එකක් වෙනස් වුණාම React පියවර 3ක cycle එකක් ක්‍රියාත්මක කරනවා:

  $$\text{Trigger} \rightarrow \text{Render} \rightarrow \text{Commit}$$

  ### 7.1 Trigger (ප්‍රේරණය කිරීම)

  Update එකක් අවශ්‍ය බව React ට දැනුම් දීමයි. `setCount(count + 1)` call කළ සැනින් trigger එකක් සිදුවේ.

  ### 7.2 Render (ප්‍රතිදානය කිරීම)

  React විසින් ඔයාගේ component function එක නැවත වරක් call කිරීමයි. මෙහිදී මුළු browser page එකම reload වෙන්නේ නැහැ. React කරන්නේ UI එකේ අලුත් virtual විස්තරයක් (new virtual description of UI) සාදා ගැනීමයි.

  ### 7.3 Commit (යාවත්කාලීන කිරීම)

  React විසින් සැබෑ browser DOM එක update කරන පියවරයි. මෙහිදී React කලින් තිබුණු UI එක සහ අලුත් UI එක සසඳා බලා (compare), වෙනස් වූ කොටස පමණක් සැබෑ DOM එකේ update කරයි (උදාහරණයක් විදිහට, count එක 0 සිට 1 දක්වා වෙනස් වුණොත්, වෙනස් වෙන්නේ ඒ අදාළ text එක විතරයි).

8. Virtual DOM in Simple Words (Virtual DOM එක සරලව)

- **Concept Explanation (Sinhala):** Virtual DOM කියන්නේ React විසින් memory එකේ තියාගන්නා UI එකේ සරල සිතියමක් වැනි description එකක්. අපි සාමාන්‍ය JavaScript වලින් වගේ `document.querySelector("#count").textContent = count` ලෙස කෙලින්ම DOM එක වෙනස් කරන්නේ නැහැ. අපි කරන්නේ JSX වලින් UI එක විස්තර කිරීම පමණයි, ඉතිරි සියලුම යාවත්කාලීන කිරීම් React විසින් Virtual DOM එක හරහා කාර්යක්ෂමව සිදුකරනු ලබනවා.

9. State Is Private to Each Component Instance (State යනු එක් එක් Component එකට පෞද්ගලික දෙයකි)

- **Concept Explanation (Sinhala):** අපි එකම component එක page එකේ දෙපාරක් පාවිච්චි කරොත්, ඒ හැම component එකකටම තමන්ගේම කියලා වෙන වෙනම **private state** එකක් ලැබෙනවා. එකක් click කරද්දී අනෙක වෙනස් වෙන්නේ නැහැ.
- **Code Block:**

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button type="button" onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}

export default function App() {
  return (
    <main>
      <Counter />
      <Counter />
    </main>
  );
}
```

10. Hook Rule: Call Hooks at the Top Level (Hooks භාවිතා කිරීමේ රීතිය)

- **Concept Explanation (Sinhala):** `useState` වැනි Hooks හැමවෙලේම component එකක **top level** (ඉහළම මට්ටමේ) පමණක් call කළ යුතුය.

  වැදගත්: කිසිම විටක Hooks පහත දේවල් ඇතුළත call කරන්න එපා:
  - `if` statements (conditions) ඇතුළේ
  - Loops (for/while) ඇතුළේ
  - Nested functions ඇතුළේ
  - Event handlers ඇතුළේ

  React එක හැම render එකකදීම Hooks ධාවනය වන අනුපිළිවෙල (order) track කරගන්නා නිසා, කොන්දේසි මත Hooks ධාවනය වීමෙන් React හට state එක පටලවා ගැනීමට ඉඩ ඇත.

11. Updating State Based on Previous State (පෙර State එක පදනම් කරගෙන Update කිරීම)

- **Concept Explanation (Sinhala):** සරලව `setCount(count + 1)` ලිවිය හැකි වුවත්, ඊළඟ state එක කලින් state එක මත සෘජුවම රඳා පවතිනවා නම් වඩාත්ම ආරක්ෂිත ක්‍රමය වන්නේ **updater function** එකක් භාවිත කිරීමයි: `setCount((prevCount) => prevCount + 1);` මෙහිදී React විසින් පෙර තිබූ නිවැරදිම අගය (`prevCount`) අපිට ලබාදෙන අතර, අප එයට 1ක් එකතු කර ආපසු ලබාදේ.
- **Code Block:**

```
import { useState } from "react";

export default function Counter() {
  const [count, setCount] = useState(0);

  function handleIncrease() {
    setCount((prevCount) => prevCount + 1);
  }

  function handleDecrease() {
    setCount((prevCount) => prevCount - 1);
  }

  function handleReset() {
    setCount(0);
  }

  return (
    <main>
      <h1>Counter</h1>
      <p>Count: {count}</p>
      <button type="button" onClick={handleDecrease}>-</button>
      <button type="button" onClick={handleIncrease}>+</button>
      <button type="button" onClick={handleReset}>Reset</button>
    </main>
  );
}
```

12. Object State (State එකක් ලෙස Object භාවිතා කිරීම)

- **Concept Explanation (Sinhala):** සමහර වෙලාවට එකිනෙකට සම්බන්ධ දත්ත කිහිපයක් එකට තබා ගැනීමට Object එකක් state එක ලෙස පාවිච්චි කරන්න පුළුවන්. උදාහරණයක් ලෙස user කෙනෙකුගේ නම, වයස, සහ නගරය එකම object එකක තැබීම දැක්විය හැකියි.
- **Code Block:**

```
const [user, setUser] = useState({
  name: "Lina",
  age: 20,
  city: "Colombo",
});
```

13. Object State Mistake: Replacing the Whole Object (Object එකක් Update කිරීමේදී සිදුවන පොදු වැරැද්දක්)

- **Concept Explanation (Sinhala):** ඔයා ළඟ ඉහත සඳහන් කළ `user` object එක තියෙද්දී, නම පමණක් වෙනස් කිරීමට සිතා `setUser({ name: "Sangeeth" });` ලෙස ලිව්වොත් මුළු object එකම අලුත් එකකින් ප්‍රතිස්ථාපනය වේ. එවිට එහි තිබූ `age` සහ `city` යන fields සදහටම මැකී යයි.

14. Correct Object State Update With Spread (Spread Operator මඟින් නිවැරදිව Object Update කිරීම)

- **Concept Explanation (Sinhala):** අගයන් නැති කර නොගෙන නිවැරදිව update කිරීමට නම්, පළමුව පරණ object එක spread operator (`...`) මඟින් copy කර, ඉන්පසු අවශ්‍ය field එක පමණක් වෙනස් කළ යුතුය.

  `setUser((prevUser) => ({ ...prevUser, name: "Sangeeth" }));`
  - `...prevUser` : පරණ user ගේ සියලුම දත්ත copy කරයි.
  - `name: "Sangeeth"` : name field එක පමණක් අලුත් අගයෙන් වෙනස් කරයි.

15. Object State With Form Inputs (Form Inputs සමඟ Object State භාවිතය)

- **Concept Explanation (Sinhala):** එකම `handleChange` function එකක් භාවිතයෙන් form එකක inputs කිහිපයක් එකම state object එකකට යාවත්කාලීන කරන ආකාරය මෙන්න:
- **Code Block:**

```
import { useState } from "react";

export default function ProfileEditor() {
  const [user, setUser] = useState({
    name: "Lina",
    age: "20",
    city: "Colombo",
  });

  function handleChange(event) {
    const { name, value } = event.target;
    setUser((prevUser) => ({
      ...prevUser,
      [name]: value,
    }));
  }

  return (
    <main>
      <h1>Profile Editor</h1>
      <label htmlFor="name">Name</label>
      <input
        id="name"
        name="name"
        value={user.name}
        onChange={handleChange}
      />

      <label htmlFor="age">Age</label>
      <input
        id="age"
        name="age"
        value={user.age}
        onChange={handleChange}
      />

      <label htmlFor="city">City</label>
      <input
        id="city"
        name="city"
        value={user.city}
        onChange={handleChange}
      />

      <section>
        <h2>Preview</h2>
        <p>Name: {user.name}</p>
        <p>Age: {user.age}</p>
        <p>City: {user.city}</p>
      </section>
    </main>
  );
}
```

16. Computed Property Names: `[name]: value` (Computed Properties විස්තරය)

- **Concept Explanation (Sinhala):** `[name]: value` මඟින් සිදුවන්නේ input එකේ ඇති HTML `name` attribute එක (උදා: `name="city"`) dynamic key එකක් ලෙස ගෙන, අදාළ state object එකෙහි එම key එකෙහි අගය පමණක් update කිරීමයි. මෙයින් එකම එක `handleChange` function එකකින් inputs සිය ගණනක් වුවද පහසුවෙන් handle කළ හැක.

17. Array State (State එකක් ලෙස Array පාවිච්චි කිරීම)

- **Concept Explanation (Sinhala):** React වලදී state එකේ ඇති arrays හැමවිටම **immutable** (වෙනස් කළ නොහැකි) ලෙස සැලකිය යුතුය. එනම්, පරණ array එක කෙලින්ම edit කරන්නේ නැතිව හැමවිටම අලුත්ම array එකක් සාදා React හට ලබාදිය යුතුය.

18. Adding to Array State (Array එකකට අලුත් අගයක් එකතු කිරීම)

- **Concept Explanation (Sinhala):** - වැරදි ක්‍රමය: `skills.push("React")` (මෙය පරණ array එක සෘජුවම වෙනස් කරයි).
  - **නිවැරදි ක්‍රමය:** `setSkills((prevSkills) => [...prevSkills, "React"]);` (පරණ elements copy කර අගට "React" එකතු කර අලුත් array එකක් සාදයි).

19. Removing From Array State (Array එකකින් අගයක් ඉවත් කිරීම)

- **Concept Explanation (Sinhala):** Array එකකින් යම් අයිතමයක් ඉවත් කිරීමට **filter()** භාවිතා කරයි. `filter()` මඟින් පරණ array එක වෙනස් නොකර, කොන්දේසියට ගැළපෙන දේ පමණක් අඩංගු අලුත්ම array එකක් සාදා දෙයි.
- **Code Block:**

```
setSkills((prevSkills) =>
  prevSkills.filter((skill) => skill !== "CSS")
);
```

20. Updating One Item in Array State (Array එකක එක් අයිතමයක් පමණක් Update කිරීම)

- **Concept Explanation (Sinhala):** Array එකක් ඇතුළේ තියෙන එක object එකක් පමණක් වෙනස් කිරීමට **map()** භාවිතා කරයි.
- **Code Block:**

```
const [items, setItems] = useState([
  { id: "i1", text: "Shoes", packed: false },
  { id: "i2", text: "Charger", packed: false },
]);

function togglePacked(id) {
  setItems((prevItems) =>
    prevItems.map((item) =>
      item.id === id ? { ...item, packed: !item.packed } : item
    )
  );
}
```

21. Why Direct Mutation Is a Problem (කෙලින්ම State වෙනස් කිරීම ප්‍රශ්නයක් වෙන්නේ ඇයි?)

- **Concept Explanation (Sinhala):** අපි `items.push("B")` වැනි ක්‍රම මඟින් array එක කෙලින්ම වෙනස් කර `setItems(items)` ලෙස ලබා දුනහොත්, React විසින් දකින්නේ පරණ array reference එකමය (මොකද memory address එක වෙනස් වෙලා නැහැ). එබැවින් React සිතන්නේ කිසිදු වෙනසක් සිදු වී නැති බවයි. එම නිසා screen එක re-render වෙන්නේ නැත.

  රීතිය: කිසිවිටක state එක කෙලින්ම වෙනස් කරන්න එපා. සැමවිටම අලුත් value එකක් සාදා එය setter function එකට දෙන්න.

22. Mutation Methods to Avoid in State (මඟහැරිය යුතු Array Methods)

- **Concept Explanation (Sinhala):** State updates වලදී පහත සඳහන්, පරණ array එක කෙලින්ම වෙනස් කරන (mutate කරන) methods භාවිතයෙන් වළකින්න:
  - `push()` , `pop()` , `shift()` , `unshift()` , `splice()` , `sort()` , `reverse()`

  ආරක්ෂිත විකල්ප ක්‍රම:
  - Item එකක් එකතු කිරීමට: `spread [...]`
  - Item එකක් ඉවත් කිරීමට: `filter()`
  - Item එකක් update කිරීමට: `map()`
  - Sort කිරීමට පෙර copy කිරීමක් කරන්න: `[...items].sort()`
  - Reverse කිරීමට පෙර copy කිරීමක් කරන්න: `[...items].reverse()`

23. Derived Values: Do Not Store What You Can Calculate (Derived Values - අනවශ්‍ය ලෙස State නොතැබීම)

- **Concept Explanation (Sinhala):** දැනට තියෙන state එකකින් ගණනය කර ලබාගත හැකි දේවල් සඳහා වෙනමම `useState` එකක් තබා නොගත යුතුය. උදාහරණයක් ලෙස, මුළු items ප්‍රමාණය සහ packed කරන ලද items ප්‍රමාණය, `items` array එකෙන් කෙලින්ම ගණනය කරගත හැක. `const packedCount = items.filter((item) => item.packed).length;` (මීට වෙනම state එකක් අවශ්‍ය නොවේ).

24. Lifting State Up (State එක ඉහළට එසවීම)

- **Concept Explanation (Sinhala):** එකම මට්ටමේ පවතින sibling components දෙකකට එකම state එකක් පොදුවේ පාවිච්චි කිරීමට අවශ්‍ය වූ විට, එම state එක ඒවායේ පොදු parent component එක වෙත ගෙන යා යුතුය. මෙයට **Lifting State Up** යැයි කියනු ලැබේ.
- **Code Block:**

```
import { useState } from "react";

function SearchInput({ query, onQueryChange }) {
  return (
    <input
      value={query}
      onChange={(event) => onQueryChange(event.target.value)}
      placeholder="Search..."
    />
  );
}

function SearchPreview({ query }) {
  return <p>You searched for: {query}</p>;
}

export default function App() {
  const [query, setQuery] = useState("");

  return (
    <main>
      <h1>Search</h1>
      <SearchInput query={query} onQueryChange={setQuery} />
      <SearchPreview query={query} />
    </main>
  );
}
```

25. useReducer Beginner Introduction (useReducer හැඳින්වීම)

- **Concept Explanation (Sinhala):** `useState` යනු සරල state සඳහාය. නමුත් state එකක් වෙනස් වීමට ක්‍රමවේද (actions) රැසක් පවතින විට (උදා: increment, decrement, reset), එම state logic එක වඩාත් සංවිධානාත්මකව තබා ගැනීමට `useReducer` භාවිත කළ හැක.

  Reducer යනු දැනට පවතින state එක සහ සිදුකළ යුතු action එක ලබාගෙන, ඊළඟ අලුත් state එක නිපදවා දෙන සරල function එකකි.

26. useReducer Parts (useReducer හි ප්‍රධාන කොටස්)

- **Concept Explanation (Sinhala):** - `state` : වර්තමාන data අගය.
  - `dispatch` : action එකක් reducer වෙත යැවීමට (send කිරීමට) භාවිතා කරන function එක.
  - `action` : සිදුවූයේ කුමක්දැයි විස්තර කරන object එකක් (බොහෝවිට `{ type: "action_name" }` ලෙස පවතී).
  - `reducer` : action එක අනුව ඊළඟ state එක තීරණය කරන function එක.
  - `initialState` : ආරම්භක අගය.

  ලියන ආකාරය: `const [state, dispatch] = useReducer(reducer, initialState);`

27. Counter With useReducer (useReducer මඟින් Counter එකක් හදන හැටි)

- **Concept Explanation (Sinhala):** පහත කේතයෙන් `useReducer` භාවිතයෙන් සාදන ලද Counter එකක් පෙන්වයි. මෙහි dispatch මඟින් action types යවා reducer එක හරහා state එක ආරක්ෂිතව update කරගනී.
- **Code Block:**

```
import { useReducer } from "react";

const initialState = {
  count: 0,
};

function reducer(state, action) {
  if (action.type === "increment") {
    return { count: state.count + 1 };
  }
  if (action.type === "decrement") {
    return { count: state.count - 1 };
  }
  if (action.type === "reset") {
    return initialState;
  }
  return state;
}

export default function CounterReducer() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <main>
      <h1>Reducer Counter</h1>
      <p>Count: {state.count}</p>
      <button type="button" onClick={() => dispatch({ type: "decrement"
})}>-</button>
      <button type="button" onClick={() => dispatch({ type: "increment"
})}>+</button>
      <button type="button" onClick={() => dispatch({ type: "reset"
})}>Reset</button>
    </main>
  );
}
```

28. How useReducer Works (useReducer ක්‍රියාත්මක වන ආකාරය)

- **Concept Explanation (Sinhala):** User කෙනෙක් button එකක් click කර `dispatch({ type: "increment" })` call කළ විට, React විසින් එම action එක සහ වර්තමාන state එක reducer function එක වෙත යවයි. Reducer එක මඟින් ලැබුණු action type එක පරීක්ෂා කර නව state object එකක් return කරයි.

  රීතිය: Reducer එක සැමවිටම අලුත් state එකක් return කළ යුතුය, පරණ state එක mutate නොකළ යුතුය.

29. When to Use useState vs useReducer (useState සහ useReducer තෝරාගන්නේ කෙසේද?)

- **Concept Explanation (Sinhala): - useState පාවිච්චි කරන්න:** State එක සරල වන විට (උදා: තනි input text එකක්, loading boolean එකක්, selected tab එකක්).
  - **useReducer පාවිච්චි කරන්න:** එකිනෙකට සම්බන්ධ actions රැසක් ඇති විට සහ එකම state එකකට නීති රැසක් ඔස්සේ update සිදුවන විට (උදා: Cart එකකට බඩු එකතු කිරීම/ඉවත් කිරීම/ප්‍රමාණ වෙනස් කිරීම).

30. Main Sprint 05 Project: Packing List (ප්‍රධාන ව්‍යාපෘතිය - Packing List App)

- **Concept Explanation (Sinhala):** මෙම ව්‍යාපෘතිය මඟින් Sprint 05 හි ඉගෙන ගත් useState, controlled inputs, preventDefault, array updates (using spread, filter, map), derived values, සහ conditional rendering යන සියල්ලම එකවර ප්‍රායෝගිකව අත්හදා බැලිය හැක.
- **Code Block:**

```
import { useState } from "react";

export default function PackingList() {
  const [newItem, setNewItem] = useState("");
  const [items, setItems] = useState([
    { id: "i1", text: "Passport", packed: false },
    { id: "i2", text: "Charger", packed: true },
  ]);

  function handleSubmit(event) {
    event.preventDefault();
    const trimmedItem = newItem.trim();
    if (trimmedItem === "") {
      return;
    }
    const item = {
      id: crypto.randomUUID(),
      text: trimmedItem,
      packed: false,
    };
    setItems((prevItems) => [...prevItems, item]);
    setNewItem("");
  }

  function handleRemove(id) {
    setItems((prevItems) => prevItems.filter((item) => item.id !== id));
  }

  function handleTogglePacked(id) {
    setItems((prevItems) =>
      prevItems.map((item) =>
        item.id === id ? { ...item, packed: !item.packed } : item
      )
    );
  }

  const packedCount = items.filter((item) => item.packed).length;

  return (
    <main>
      <h1>Packing List</h1>
      <form onSubmit={handleSubmit}>
        <label htmlFor="newItem">New item</label>
        <input
          id="newItem"
          value={newItem}
          onChange={(event) => setNewItem(event.target.value)}
        />
        <button type="submit" disabled={newItem.trim() === ""}>
          Add
        </button>
      </form>
      <p>
        Packed {packedCount} of {items.length}
      </p>
      <section>
        <h2>Items</h2>
        {items.length === 0 ? (
          <p>No items yet.</p>
        ) : (
          <ul>
            {items.map((item) => (
              <li key={item.id}>
                <label>
                  <input
                    type="checkbox"
                    checked={item.packed}
                    onChange={() => handleTogglePacked(item.id)}
                  />
                  {item.text}
                </label>
                <button type="button" onClick={() =>
handleRemove(item.id)}>
                  Remove
                </button>
              </li>
            ))}
          </ul>
        )}
      </section>
    </main>
  );
}
```

31. Packing List Explanation (Packing List App එකෙහි ක්‍රියාකාරීත්වය සරලව)

- **Concept Explanation (Sinhala): - Input එක සඳහා State එක:** `newItem` මඟින් user ටයිප් කරන දේ මතක තබා ගනී.
  - **Array එක සඳහා State එක:** `items` මඟින් බඩු ලැයිස්තුව මතක තබා ගනී.
  - **Add item:** `[...prevItems, item]` මඟින් පරණ array එකට අලුත් item එක එකතු කර අලුත් array එකක් සාදයි.
  - **Remove item:** `filter()` මඟින් තෝරාගත් item එක ඉවත් කර ඉතිරි ඒවා සහිත අලුත් array එකක් සාදයි.
  - **Toggle packed:** `map()` මඟින් අදාළ item එක පමණක් සොයා එහි `packed` අගය true/false ලෙස වෙනස් කරයි.
  - **Derived packed count:** `filter().length` මඟින් packed කරන ලද ප්‍රමාණය සෘජුවම ගණනය කරයි.

32. Sprint 05 Debug Task (කේතයේ දෝෂ සෙවීම)

- **Concept Explanation (Sinhala): - දෝෂ සහිත කේතය (Broken Code):**

  ```
  function List() {
    const [items, setItems] = useState(["A"]);
    function addItem() {
      items.push("B");
      setItems(items);
    }
    return <button onClick={addItem}>{items.length}</button>;
  }
  ```

  ප්‍රශ්න: `useState` import කර නොමැත, `items.push` මඟින් array එක සෘජුවම mutate කර ඇත, reference එක සමාන බැවින් React හට වෙනස හඳුනාගත නොහැක, button එකෙහි `type="button"` නොමැත.
  - **නිවැරදි කරන ලද කේතය (Fixed Code):**
- **Code Block:**

```
import { useState } from "react";

export default function List() {
  const [items, setItems] = useState(["A"]);

  function addItem() {
    setItems((prevItems) => [...prevItems, "B"]);
  }

  return (
    <button type="button" onClick={addItem}>
      {items.length}
    </button>
  );
}
```

33. Common Sprint 05 Mistakes (පොදු වැරදි සහ ඒවා වළක්වා ගන්නා ආකාරය)

- **Concept Explanation (Sinhala): 1. කෙලින්ම state වෙනස් කිරීම: - ❌** `count = count + 1`
  - `setCount(count + 1)`

  2. **Object එකක් mutate කිරීම: - ❌** `user.name = "Sangeeth"; setUser(user);`
     - `setUser((prevUser) => ({ ...prevUser, name: "Sangeeth" }));`
  3. **නොදැනුවත්වම මුළු object එකම replace කිරීම: - ❌** `setUser({ name: "Sangeeth" });` (අනෙකුත් fields මැකී යයි)
     - `setUser((prevUser) => ({ ...prevUser, name: "Sangeeth" }));`
  4. **Array එකක් push() මඟින් වෙනස් කිරීම: - ❌** `items.push(newItem); setItems(items);`
     - `setItems((prevItems) => [...prevItems, newItem]);`
  5. **Splice() මඟින් array එකකින් අයිතම ඉවත් කිරීම: - ❌** `items.splice(index, 1); setItems(items);`
     - `setItems((prevItems) => prevItems.filter((item) => item.id !== id));`
  6. **Hooks කොන්දේසි ඇතුළත call කිරීම: - ❌** `if (enabled) { const [count, setCount] = useState(0); }`
     - `const [count, setCount] = useState(0);` (හැමවිටම top level එකේ පමණක් තබන්න)

34. Revision Questions and Answers (ප්‍රශ්නෝත්තර සංශෝධනය)

- **Concept Explanation (Sinhala): - ප්‍රශ්නය 1: State යනු කුමක්ද?**
  - **පිළිතුර:** State යනු React component එකක් තුළ වෙනස් විය හැකි දත්ත මතක තබා ගැනීමට ඇති මතකයයි (memory).
  - **ප්‍රශ්නය 2:** `useState` **මඟින් ලබාදෙන්නේ කුමක්ද?**
    - **පිළිතුර:** එය අගයන් දෙකක් සහිත array එකක් ලබාදේ: 1. වර්තමාන state අගය (current state value) සහ 2. එය වෙනස් කරන setter function එක.
  - **ප්‍රශ්නය 3: Setter function එකම පාවිච්චි කළ යුත්තේ ඇයි?**
    - **පිළිතුර:** මොකද setter function එක call කළ විට පමණක් React හට state එක වෙනස් වූ බව වැටහී UI එක re-render කිරීමට පියවර ගන්නා බැවිනි.
  - **ප්‍රශ්නය 4: Trigger, Render, සහ Commit යනු මොනවාද?**
    - **පිළිතුර: Trigger** යනු update එකක් අවශ්‍ය බව React ට දැන්වීමයි. **Render** යනු component function එක නැවත ධාවනය කර නව UI විස්තරයක් සෑදීමයි. **Commit** යනු වෙනස් වූ දේ පමණක් සැබෑ browser DOM එකට ඇතුළත් කිරීමයි.
  - **ප්‍රශ්නය 5: Direct Mutation (සෘජුවම වෙනස් කිරීම) වැරදි ඇයි?**
    - **පිළිතුර:** පරණ object/array එක කෙලින්ම වෙනස් කළ විට එහි memory reference එක වෙනස් නොවන බැවින්, React හට වෙනස හඳුනාගත නොහැකි වී UI එක update නොවී පවතී.
  - **ප්‍රශ්නය 6: Object state එකක් ආරක්ෂිතව update කරන්නේ කෙසේද?**
    - **පිළිතුර:** Spread operator (`...`) මඟින් පරණ object එක copy කර, අවශ්‍ය field එක පමණක් වෙනස් කිරීමෙනි.
  - **ප්‍රශ්නය 7:** `[name]: value` **මඟින් සිදුවන්නේ කුමක්ද?**
    - **පිළිතුර:** Input එකෙහි ඇති HTML `name` attribute එක dynamic key එකක් ලෙස ගෙන අදාළ state property එක update කරයි.
  - **ප්‍රශ්නය 8: Array එකකට අලුත් item එකක් එකතු කරන්නේ කෙසේද?**
    - **පිළිතුර:** `setItems((prevItems) => [...prevItems, newItem]);`
  - **ප්‍රශ්නය 9: Array එකකින් item එකක් ඉවත් කරන්නේ කෙසේද?**
    - **පිළිතුර:** `filter()` භාවිතයෙන්: `setItems((prevItems) => prevItems.filter((item) => item.id !== id));`
  - **ප්‍රශ්නය 10: Array එකක එක item එකක් පමණක් update කරන්නේ කෙසේද?**
    - **පිළිතුර:** `map()` භාවිතයෙන්: `setItems((prevItems) => prevItems.map((item) => item.id === id ? { ...item, packed: !item.packed } : item));`
  - **ප්‍රශ්නය 11: Lifting State Up යනු කුමක්ද?**
    - **පිළිතුර:** Sibling components කිහිපයකට අවශ්‍ය පොදු state එකක් ඒවායේ ළඟම ඇති පොදු parent component එක වෙත ගෙන යාමයි.
  - **ප්‍රශ්නය 12:** `useReducer` **වඩාත් ප්‍රයෝජනවත් වන්නේ කුමන අවස්ථාවලදීද?**
    - **පිළිතුර:** State එකක් වෙනස් කිරීම සඳහා එකිනෙකට සම්බන්ධ actions රැසක් පවතින විට සහ logic එක වඩාත් පැහැදිලිව තබා ගැනීමට අවශ්‍ය වූ විටයි.

35. Mini Revision Sheet (කෙටි සාරාංශය)

- **Concept Explanation (Sinhala): - State:** React හි වෙනස් වන මතකය.
  - **Setter:** State එක update කර UI එක re-render කිරීමට යොදාගනී.
  - **Updater Function:** පෙර state එක මත ඊළඟ state එක රඳා පවතින විට යොදාගනී (`(prev) => prev + 1`).
  - **Render Cycle:** Trigger -> Render -> Commit.
  - **Object Update:** `setUser((prev) => ({ ...prev, name: "New" }));`
  - **Array Add:** `setItems((prev) => [...prev, item]);`
  - **Array Remove:** `setItems((prev) => prev.filter(item => item.id !== id));`
  - **Array Update:** `setItems((prev) => prev.map(item => item.id === id ? { ...item, checked: true } : item));`
  - **useReducer:** Action-based සංකීර්ණ state logic සඳහා සුදුසුය.